

OSSP

Skill Week - 5

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1-:

**To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.**

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/stat.h>

int main() {

    char path[PATH_MAX];
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX] = "";

    printf("Simple Directory Navigation Program\n");
    printf("Commands: cd <path>, pwd, previous, exit\n");

    while (1) {

        // Display current directory
        if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
            perror("getcwd");
            break;
        }

        printf("$$$ ");

        if (fgets(path, sizeof(path), stdin) == NULL) {
            break;
        }

        // Remove newline
        path[strcspn(path, "\n")] = '\0';

        // Exit command
        if (strcmp(path, "exit") == 0) {
            printf("Exiting...\n");
            break;
        }

        // pwd command
        if (strcmp(path, "pwd") == 0) {
            printf("Current Directory: %s\n", current_dir);
            continue;
        }

        // previous command
        if (strcmp(path, "previous") == 0) {
            if (strlen(previous_dir) == 0) {
```

```
Ubuntu × + ▾
GNU nano 8.7.1 navigation.c
    continue;
}

// previous command
if (strcmp(path, "previous") == 0) {
D   if (strlen(previous_dir) == 0) {
        printf("No previous directory available.\n");
        continue;
    }

    char temp[PATH_MAX];

    strcpy(temp, current_dir);

    if (chdir(previous_dir) == -1) {
        perror("chdir");
        continue;
    }

    strcpy(previous_dir, temp);

    printf("Changed to previous directory.\n");
    continue;
}

// cd command
if (strncmp(path, "cd ", 3) == 0) {

    char *new_path = path + 3;

    // Check if path exists
    struct stat st;

    if (stat(new_path, &st) == -1) {
        perror("Invalid path");
        continue;
    }

    // Check if it is a directory
    if (!S_ISDIR(st.st_mode)) {
        printf("Error: Path is not a directory.\n");
        continue;
    }

    // Save current directory
    strcpy(previous_dir, current_dir);

    // Change directory
```

```
Ubuntu
GNU nano 8.7.1 navigation.c *
    printf("Changed to previous directory.\n");
    continue;
}

// cd command
if (strncmp(path, "cd ", 3) == 0) {
    char *new_path = path + 3;

    // Check if path exists
    struct stat st;

    if (stat(new_path, &st) == -1) {
        perror("Invalid path");
        continue;
    }

    // Check if it is a directory
    if (!S_ISDIR(st.st_mode)) {
        printf("Error: Path is not a directory.\n");
        continue;
    }

    // Save current directory
    strcpy(previous_dir, current_dir);

    // Change directory
    if (chdir(new_path) == -1) {
        perror("chdir");
        continue;
    }

    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd");
        continue;
    }

    printf("Directory changed successfully.\n");
    printf("New Working Directory: %s\n", current_dir);
} else {
    printf("Invalid command.\n");
    printf("Use: cd <path>, pwd, previous, or exit\n");
}

return 0;
}
```

**Output:-**

```
nocturnal@nocturnal:~/producer-consumer$ gcc navigation.c -o navigation
nocturnal@nocturnal:~/producer-consumer$ gcc navigation.c -o navigation
nocturnal@nocturnal:~/producer-consumer$ ./navigation
Simple Directory Navigation Program
Commands: cd <path>, pwd, previous, exit
$$$ pwd
Current Directory: /home/nocturnal/producer-consumer
$$$ ls
Invalid command.
Use: cd <path>, pwd, previous, or exit
$$$ cd /tmp
Directory changed successfully.
New Working Directory: /tmp
$$$ pwd
Current Directory: /tmp
$$$ previous
Changed to previous directory.
$$$ exit
Exiting...
nocturnal@nocturnal:~/producer-consumer$ D
```

Task-2:

To Identify Built-ins, Create Dispatch Table, Execute In Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_ARGS 20

/* Function declarations */
void command_pwd(char **args);
void command_cd(char **args);
void command_help(char **args);
void command_exit(char **args);

/* Dispatch table */
typedef struct {
    char *name;
    void (*function)(char **args);
} Command;

/* Built-in command table */
Command dispatch_table[] = {
    {"pwd", command_pwd},
    {"cd", command_cd},
    {"help", command_help},
    {"exit", command_exit}
};

int command_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);

/* pwd command */
void command_pwd(char **args) {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
    } else {
        perror("pwd");
    }
}

/* cd command */
void command_cd(char **args) {
    if (args[1] == NULL) {
        printf("cd: missing argument\n");
    }
}
```

```
Ubuntu × + ▾
GNU nano 8.7.1 Shell.c *
void command_cd(char **args) {
    if (args[1] == NULL) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args[1]) != 0) {
        perror("cd");
    }
}

/* help command */
void command_help(char **args) {
    printf("\nAvailable commands:\n");
    printf("  pwd      - Show current directory\n");
    printf("  cd <dir> - Change directory\n");
    printf("  help     - Show available commands\n");
    printf("  exit     - Exit shell\n");
    printf("  Other commands are executed as processes.\n\n");
}

/* exit command */
void command_exit(char **args) {
    printf("Exiting shell...\n");
    exit(0);
}

/* Execute external process */
void execute_process(char **args) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        if (execvp(args[0], args) == -1) {
            perror("Command execution failed");
            exit(1);
        }
    } else {

```

```
Ubuntu × + ▾
GNU nano 8.7.1 Shell.c *
    }
    } else {
        /* Parent process */
        waitpid(pid, NULL, 0);
    }
}

/* Check dispatch table */
int execute_builtin(char **args) {
    for (int i = 0; i < command_count; i++) {
        if (strcmp(args[0], dispatch_table[i].name) == 0) {
            dispatch_table[i].function(args);
            return 1;
        }
    }
    return 0;
}

/* Convert input into arguments */
int parse_input(char *input, char **args) {
    int count = 0;
    char *token = strtok(input, " \t\n");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t\n");
    }
    args[count] = NULL;
    return count;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    D
```



```
Ubuntu
GNU nano 8.7.1 Shell.c *
    token = strtok(NULL, " \t\n");
}

args[count] = NULL;

return count;
}

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("=====\n");
    printf("      SIMPLE COMMAND SHELL      \n");
    printf("=====\\n");

    while (1) {
        printf("myshell> ");

        /* Read command */
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        /* Remove newline */
        input[strcspn(input, "\\n")] = '\\0';

        /* Ignore empty input */
        if (strlen(input) == 0) {
            continue;
        }

        /* Convert input into arguments */
        parse_input(input, args);

        /* Check built-in commands */
        if (execute_builtin(args)) {
            continue;
        }

        /* Execute external command */
        execute_process(args);
    }

    return 0;
}
```

## Output –

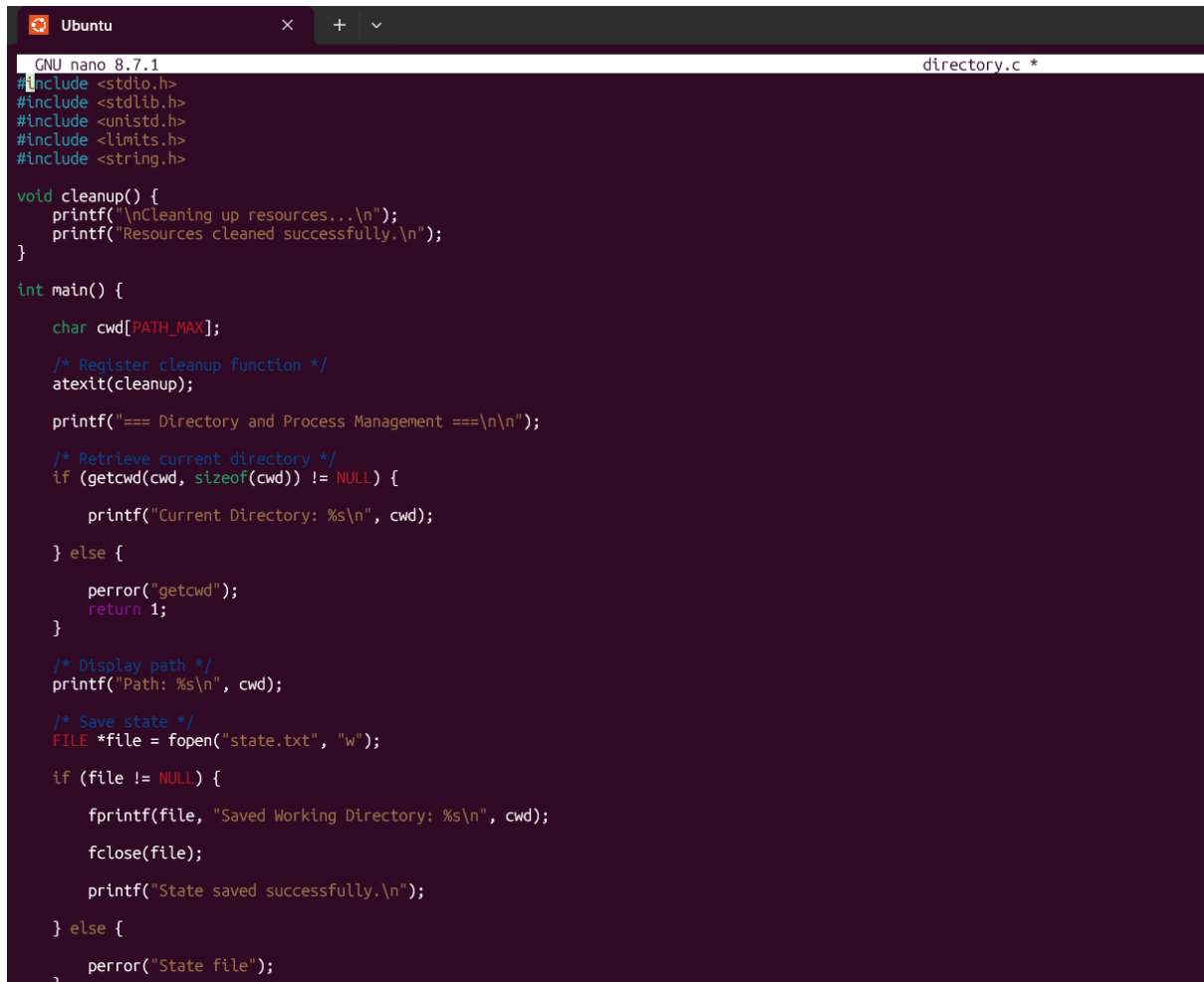
```
Ubuntu
nocturnal@nocturnal:~/producer-consumer$ gcc Shell.c -o Shell
nocturnal@nocturnal:~/producer-consumer$ ./Shell
=====
SIMPLE COMMAND SHELL
=====
myshell> pwd
/home/nocturnal/producer-consumer
myshell> ls
Commands Shell buffer.c consumer.c main.c navigation.c pipeline pipeline_structure producer.c producer_consumer redirection.c sync.h
Commands.c Shell.c buffer.h consumer.h navigation output.txt pipeline.c pipeline_structure.c producer.h redirection sync.c
myshell> help

Available commands:
  pwd      - Show current directory
  cd <dir> - Change directory
  help     - Show available commands
  exit     - Exit shell
Other commands are executed as processes.

myshell> tmp
Command execution failed: No such file or directory
myshell> cd tmp
cd: No such file or directory
myshell> exit
Exiting shell...
```

Task 3 - :

**To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.**



```
GNU nano 8.7.1 directory.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

void cleanup() {
    printf("\nCleaning up resources...\n");
    printf("Resources cleaned successfully.\n");
}

int main() {
    char cwd[PATH_MAX];

    /* Register cleanup function */
    atexit(cleanup);

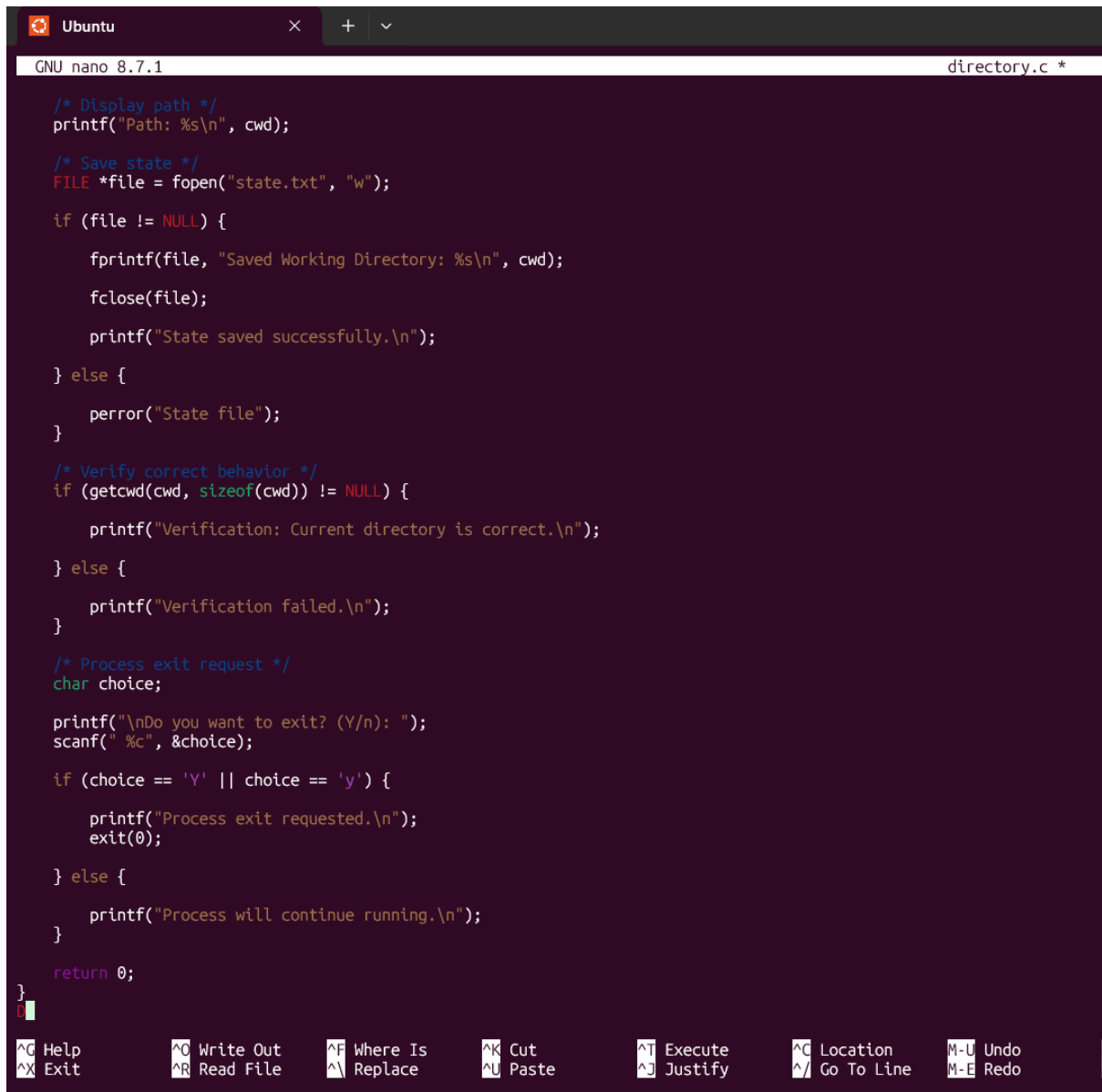
    printf("=== Directory and Process Management ===\n\n");

    /* Retrieve current directory */
    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("Current Directory: %s\n", cwd);
    } else {
        perror("getcwd");
        return 1;
    }

    /* Display path */
    printf("Path: %s\n", cwd);

    /* Save state */
    FILE *file = fopen("state.txt", "w");

    if (file != NULL) {
        fprintf(file, "Saved Working Directory: %s\n", cwd);
        fclose(file);
        printf("State saved successfully.\n");
    } else {
        perror("State file");
    }
}
```



```
GNU nano 8.7.1 directory.c *

/* Display path */
printf("Path: %s\n", cwd);

/* Save state */
FILE *file = fopen("state.txt", "w");

if (file != NULL) {

    fprintf(file, "Saved Working Directory: %s\n", cwd);

    fclose(file);

    printf("State saved successfully.\n");

} else {

    perror("State file");

}

/* Verify correct behavior */
if (getcwd(cwd, sizeof(cwd)) != NULL) {

    printf("Verification: Current directory is correct.\n");

} else {

    printf("Verification failed.\n");

}

/* Process exit request */
char choice;

printf("\nDo you want to exit? (Y/n): ");
scanf(" %c", &choice);

if (choice == 'Y' || choice == 'y') {

    printf("Process exit requested.\n");
    exit(0);

} else {

    printf("Process will continue running.\n");

}

return 0;
}
```

^G Help   ^O Write Out   ^F Where Is   ^K Cut   ^T Execute   ^C Location   M-U Undo  
^X Exit   ^R Read File   ^\ Replace   ^U Paste   ^J Justify   ^\_ Go To Line   M-E Redo

Output - :

```
nocturnal@nocturnal:~/producer-consumer$ nano directory.c
nocturnal@nocturnal:~/producer-consumer$ gcc directory.c -o directory
nocturnal@nocturnal:~/producer-consumer$ gcc directory.c -o directory
nocturnal@nocturnal:~/producer-consumer$ ./directory
=== Directory and Process Management ===

Current Directory: /home/nocturnal/producer-consumer
Path: /home/nocturnal/producer-consumer
State saved successfully.
Verification: Current directory is correct.

Do you want to exit? (Y/n): Y
Process exit requested.

Cleaning up resources...
Resources cleaned successfully.
nocturnal@nocturnal:~/producer-consumer$ D
```

**Task 4 -: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.**

```
GNU nano 8.7.1 export.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_NAME 100
#define MAX_VALUE 100

/* Validate environment variable name */
int valid_name(char *name)
{
    if (name == NULL || name[0] == '\0')
        return 0;

    /* First character must be letter or underscore */
    if (!isalpha(name[0]) && name[0] != '_')
        return 0;

    /* Remaining characters */
    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum(name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

/* Process export command */
void export_variable(char *input)
{
    char name[MAX_NAME];
    char value[MAX_VALUE];

    /* Find '=' */
    char *equal = strchr(input, '=');

    if (equal == NULL)
    {
        printf("Error: Invalid export syntax.\n");
        printf("Usage: export NAME=VALUE\n");
        return;
    }

    /* Extract variable name */
    int name_length = equal - input;
```

```
Ubuntu × + ▾
GNU nano 8.7.1 export.c *

/* Process export command */
void export_variable(char *input)
{
    char name[MAX_NAME];
    char value[MAX_VALUE];

    /* Find '=' */
    char *equal = strchr(input, '=');

    if (equal == NULL)
    {
        printf("Error: Invalid export syntax.\n");
        printf("Usage: export NAME=VALUE\n");
        return;
    }

    /* Extract variable name */
    int name_length = equal - input;

    if (name_length <= 0 || name_length >= MAX_NAME)
    {
        printf("Error: Invalid variable name.\n");
        return;
    }

    strncpy(name, input, name_length);
    name[name_length] = '\0';

    /* Extract value */
    strcpy(value, equal + 1);

    /* Validate variable name */
    if (!valid_name(name))
    {
        printf("Error: Invalid variable name: %s\n", name);
        return;
    }

    /* Check whether variable already exists */
    char *old_value = getenv(name);

    if (old_value != NULL)
    {
        printf("Existing variable found: %s=%s\n", name, old_value);
    }

    /* Set environment variable */
    if (setenv(name, value, 1) == -1)
```

```
Ubuntu × + ▾
GNU nano 8.7.1 export.c *
/* Check whether variable already exists */
char *old_value = getenv(name);

if (old_value != NULL)
{
    printf("Existing variable found: %s=%s\n", name, old_value);
}

/* Set environment variable */
if (setenv(name, value, 1) == -1)
{
    perror("setenv");
    return;
}

printf("Export successful: %s=%s\n", name, value);
}

/* Test child process */
void test_child_process()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        printf("\nChild Process:\n");

        char *value = getenv("MYVAR");

        if (value != NULL)
            printf("MYVAR=%s\n", value);
        else
            printf("MYVAR is not set.\n");

        exit(0);
    }
    else
    {
        wait(NULL);

        printf("Child process completed.\n");
    }
}
```

```
int main()
{
    char input[MAX_INPUT];

    printf("=====\n");
    printf("      EXPORT COMMAND TESTER\n");
    printf("=====\n\n");

    while (1)
    {
        printf("shell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        /* Remove newline */
        input[strcspn(input, "\n")] = '\0';

        /* Exit */
        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting...\n");
            break;
        }

        /* Export command */
        if (strncmp(input, "export ", 7) == 0)
        {
            export_variable(input + 7);
        }

        /* Test child process */
        else if (strcmp(input, "test") == 0)
        {
            test_child_process();
        }

        /* Display variable */
        else if (strncmp(input, "print ", 6) == 0)
        {
            char *name = input + 6;
            char *value = getenv(name);

            if (value != NULL)
                printf("%s=%s\n", name, value);
            else
                printf("%s is not set.\n", name);
        }
    }
}
```



```
GNU nano 8.7.1 export.c *
{
    printf("shell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Exit */
    if (strcmp(input, "exit") == 0)
    {
        printf("Exiting...\n");
        break;
    }

    /* Export command */
    if (strncmp(input, "export ", 7) == 0)
    {
        export_variable(input + 7);
    }

    /* Test child process */
    else if (strcmp(input, "test") == 0)
    {
        test_child_process();
    }

    /* Display variable */
    else if (strncmp(input, "print ", 6) == 0)
    {
        char *name = input + 6;
        char *value = getenv(name);

        if (value != NULL)
            printf("%s=%s\n", name, value);
        else
            printf("%s is not set.\n", name);
    }

    else
    {
        printf("Unknown command.\n");
    }
}

return 0;
```

Output:-

```
Exiting...
nocturnal@nocturnal:~/producer-consumer$ nano export.c
nocturnal@nocturnal:~/producer-consumer$ gcc export.c -o export
nocturnal@nocturnal:~/producer-consumer$ ./export
=====
EXPORT COMMAND TESTER
=====

shell> export MYVAR=Hello
Export successful: MYVAR=Hello

shell> test

Child Process:
MYVAR=Hello
Child process completed.

shell> █
```